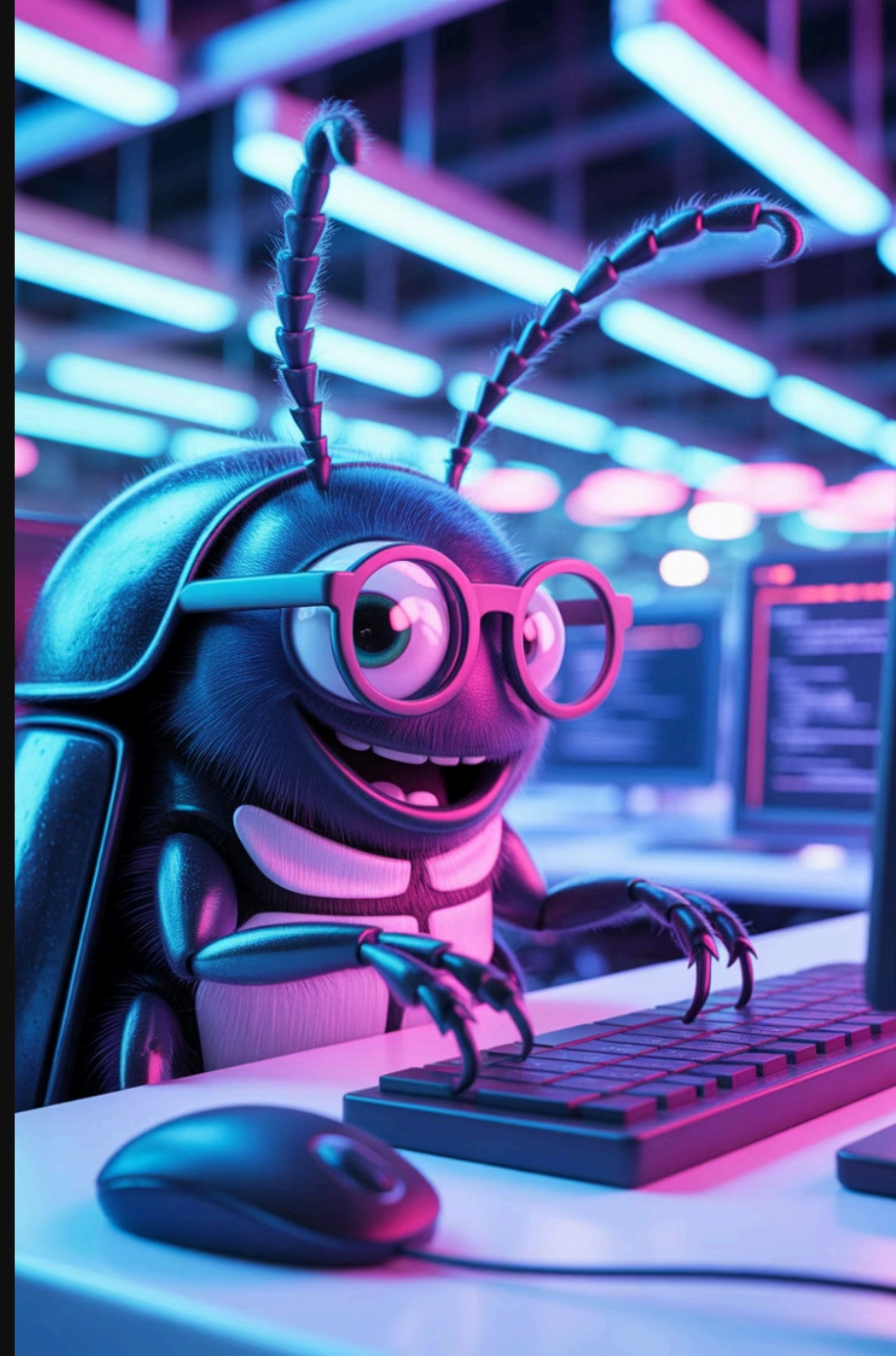


Single Test Instance: Advanced Example Overview

- This **example** demonstrates a **more complex use** of a **single test instance**.
- **Test methods** depend on each other and **share state**.
- The **scenario** involves **testing user** lifecycle **operations**.
- The **setup** goes **beyond** simple **unit tests**.

 **by Rodrigo Martins Pagliares**

Last update: 12/26/2025



User Lifecycle Under Test

The **test suite** to be **evolved** in this **example** covers four **operations**:

- **Create** user
- **Update** user
- **Get** user
- **Delete** user

Each **operation** **depends** on the **previous** one.

Correct execution order is essential!



Demo Project Context

Project Name

TestLifecyclePerClassDemonstration.

Test Class

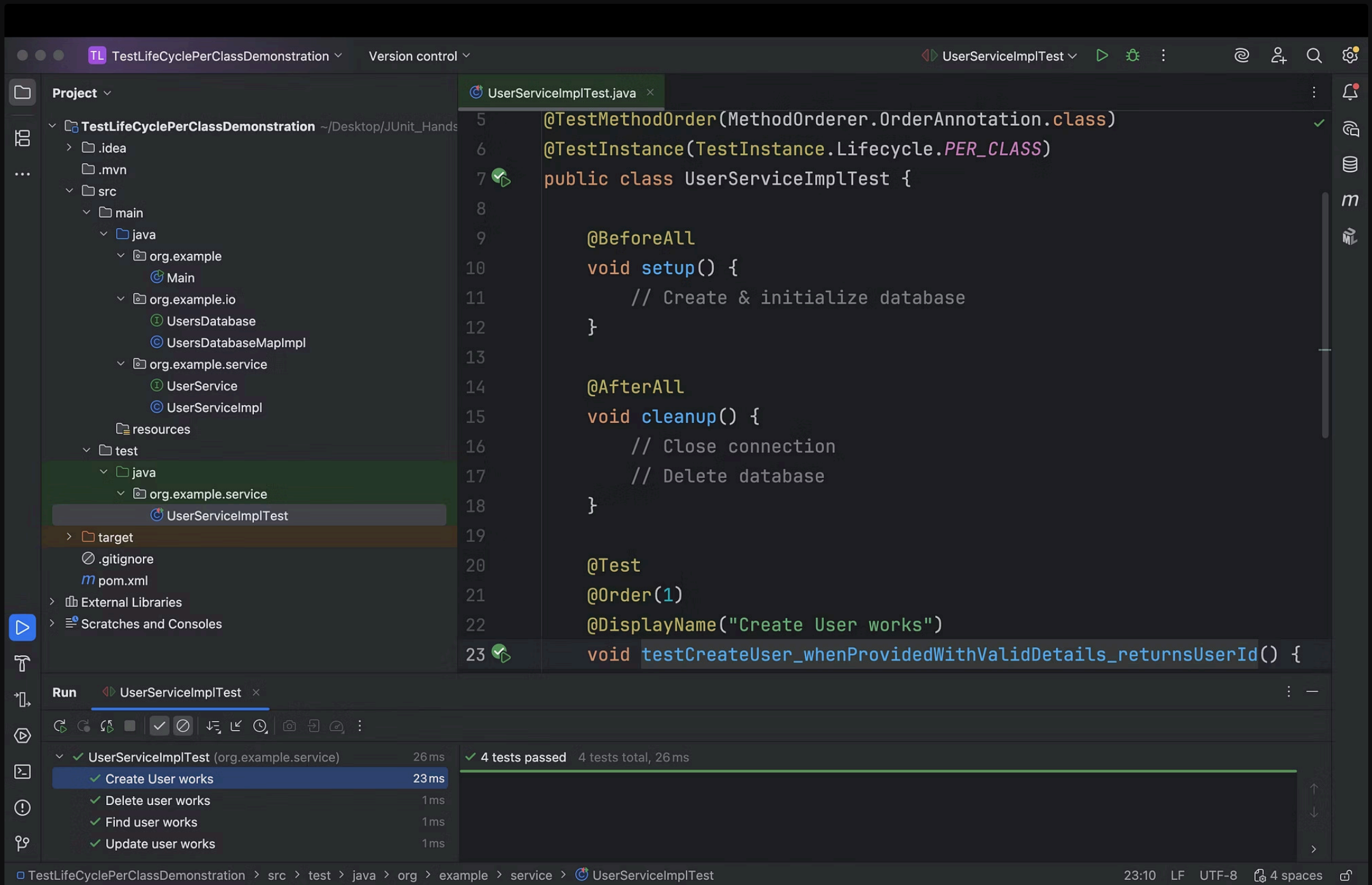
The project contains a single **test class**:

UserServiceImplementationTest.



Initial project

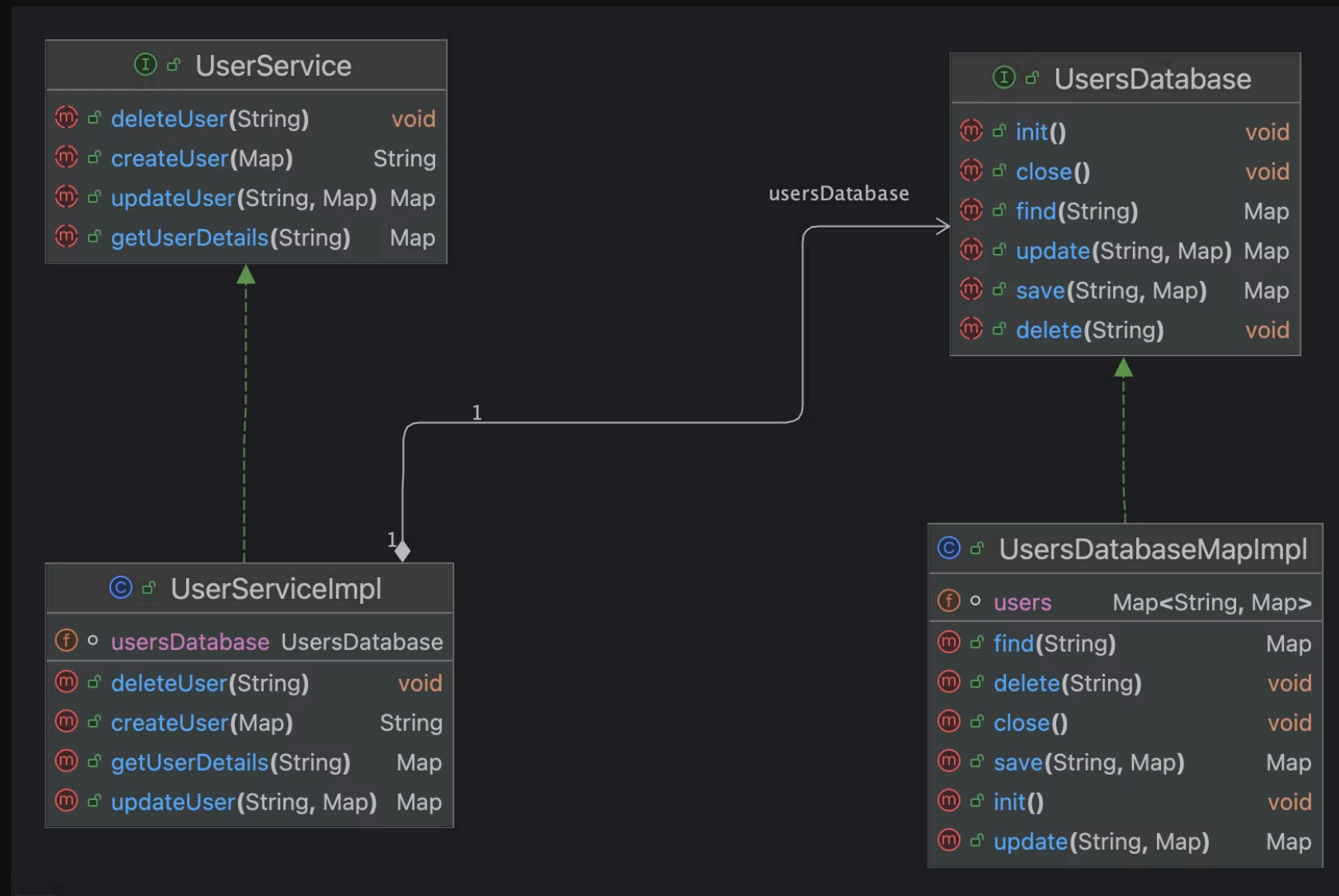
Download from the **source-code repository** an **initial project** that will be evolved in this **lesson** and the **next one**.



Maven project: TestLifeCyclePerClassDemonstration

Initial project

The **UML class diagram** depicts **classes's properties** and **relationship** in this example.

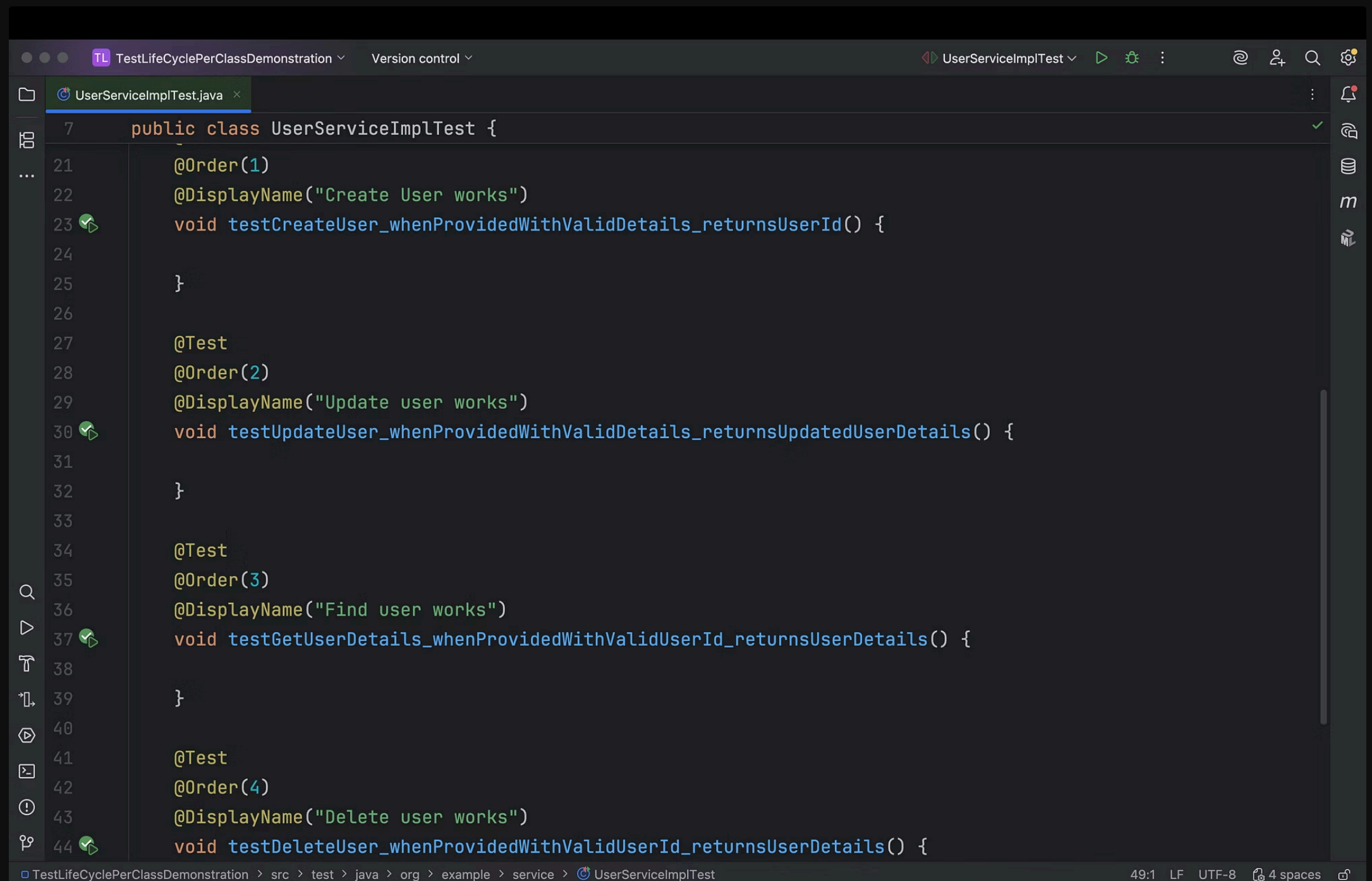


Our goal is to test **UserServiceImpl** class: **createUser**, **deleteUser**, **getUserDetails**, and **updateUser** methods.

CRUD Operations on user entity

The `UserServiceImplTest` **test class** has several **test methods** that run in order.

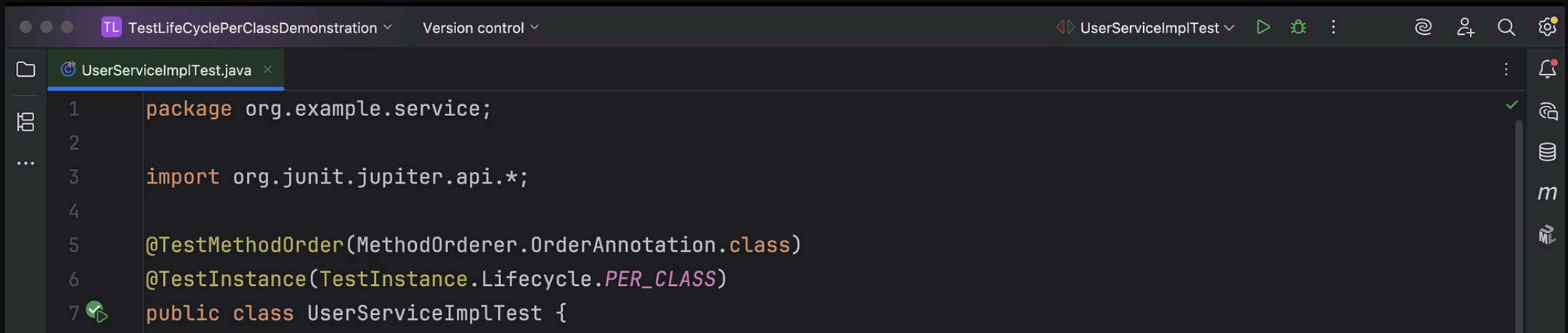
The **methods** aim at testing **Create, Retrieve, Update, and Delete operations** on a **user entity**.



```
7      public class UserServiceImplTest {  
21         @Order(1)  
22         @DisplayName("Create User works")  
23         void testCreateUser_whenProvidedWithValidDetails_returnsUserId() {  
24  
25         }  
26  
27         @Test  
28         @Order(2)  
29         @DisplayName("Update user works")  
30         void testUpdateUser_whenProvidedWithValidDetails_returnsUpdatedUserDetails() {  
31  
32         }  
33  
34         @Test  
35         @Order(3)  
36         @DisplayName("Find user works")  
37         void testGetUserDetails_whenProvidedWithValidUserId_returnsUserDetails() {  
38  
39         }  
40  
41         @Test  
42         @Order(4)  
43         @DisplayName("Delete user works")  
44         void testDeleteUser_whenProvidedWithValidUserId_returnsUserDetails() {
```

The screenshot shows an IDE window titled "TestLifeCyclePerClassDemonstration" with a sub-tab "Version control". The active file is "UserServiceImplTest.java". The code defines a public class "UserServiceImplTest" with four test methods, each annotated with "@Test", "@Order", and "@DisplayName". The methods are: "testCreateUser_whenProvidedWithValidDetails_returnsUserId()", "testUpdateUser_whenProvidedWithValidDetails_returnsUpdatedUserDetails()", "testGetUserDetails_whenProvidedWithValidUserId_returnsUserDetails()", and "testDeleteUser_whenProvidedWithValidUserId_returnsUserDetails()". The IDE interface includes a sidebar with icons for file explorer, search, and other tools, and a bottom status bar showing the file path and encoding.

Test Class Configuration



The screenshot shows an IDE window titled 'TestLifeCyclePerClassDemonstration'. The active file is 'UserServiceImplTest.java'. The code is as follows:

```
1 package org.example.service;
2
3 import org.junit.jupiter.api.*;
4
5 @TestMethodOrder(MethodOrderer.OrderAnnotation.class)
6 @TestInstance(TestInstance.Lifecycle.PER_CLASS)
7 public class UserServiceImplTest {
```

The IDE interface includes a top bar with tabs for 'TestLifeCyclePerClassDemonstration' and 'Version control'. On the right, there are icons for running tests, a search icon, and a settings icon. The left sidebar shows a file explorer with a folder icon and a search icon.

Annotation Used

The test class uses

`@TestInstance(PER_CLASS)`. (Line 6)

Instance Creation

Only one instance of the test class is created.

Shared Variables

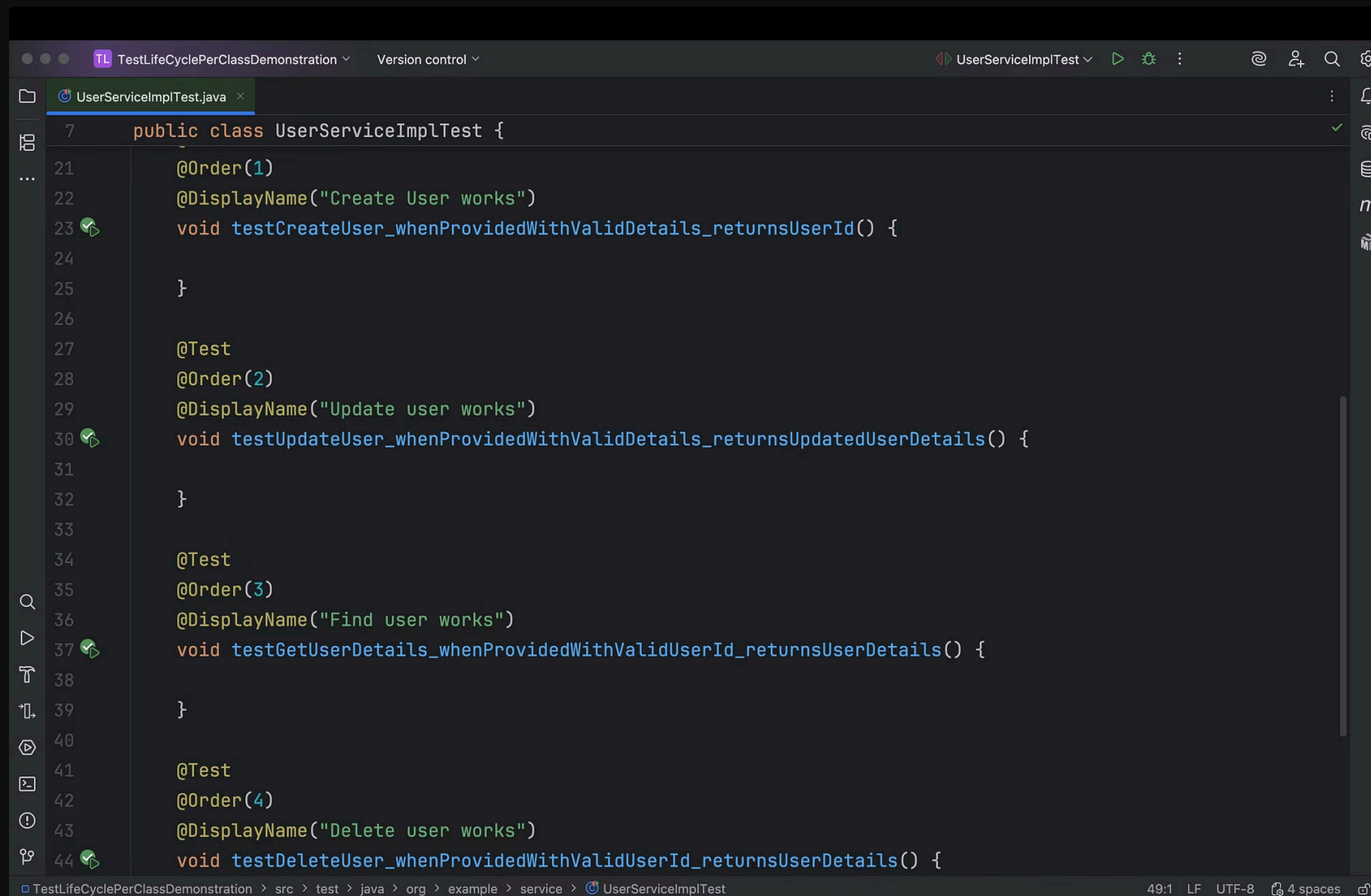
All test methods share the same instance variables.

Key Benefit

This enables state sharing across tests.

Ordered Test Method Execution

- Test methods are annotated with **@Order**.
 - Execution order is explicitly controlled.
 - create user → update → get → delete.



```
7 public class UserServiceImplTest {  
21     @Order(1)  
22     @DisplayName("Create User works")  
23     void testCreateUser_whenProvidedWithValidDetails_returnsUserId() {  
24  
25     }  
26  
27     @Test  
28     @Order(2)  
29     @DisplayName("Update user works")  
30     void testUpdateUser_whenProvidedWithValidDetails_returnsUpdatedUserDetails() {  
31  
32     }  
33  
34     @Test  
35     @Order(3)  
36     @DisplayName("Find user works")  
37     void testGetUserDetails_whenProvidedWithValidUserId_returnsUserDetails() {  
38  
39     }  
40  
41     @Test  
42     @Order(4)  
43     @DisplayName("Delete user works")  
44     void testDeleteUser_whenProvidedWithValidUserId_returnsUserDetails() {
```

Dependency Between Test Methods



Create User

The create user test **generates a user ID**.



Update User

The **update user** test **needs** that **user ID**.



Subsequent Tests

Subsequent tests rely on the **same user data**.

create user → **update** → **get** → **delete**.

Shared state is **required** for **correctness**.



Lifecycle Methods with PER_CLASS

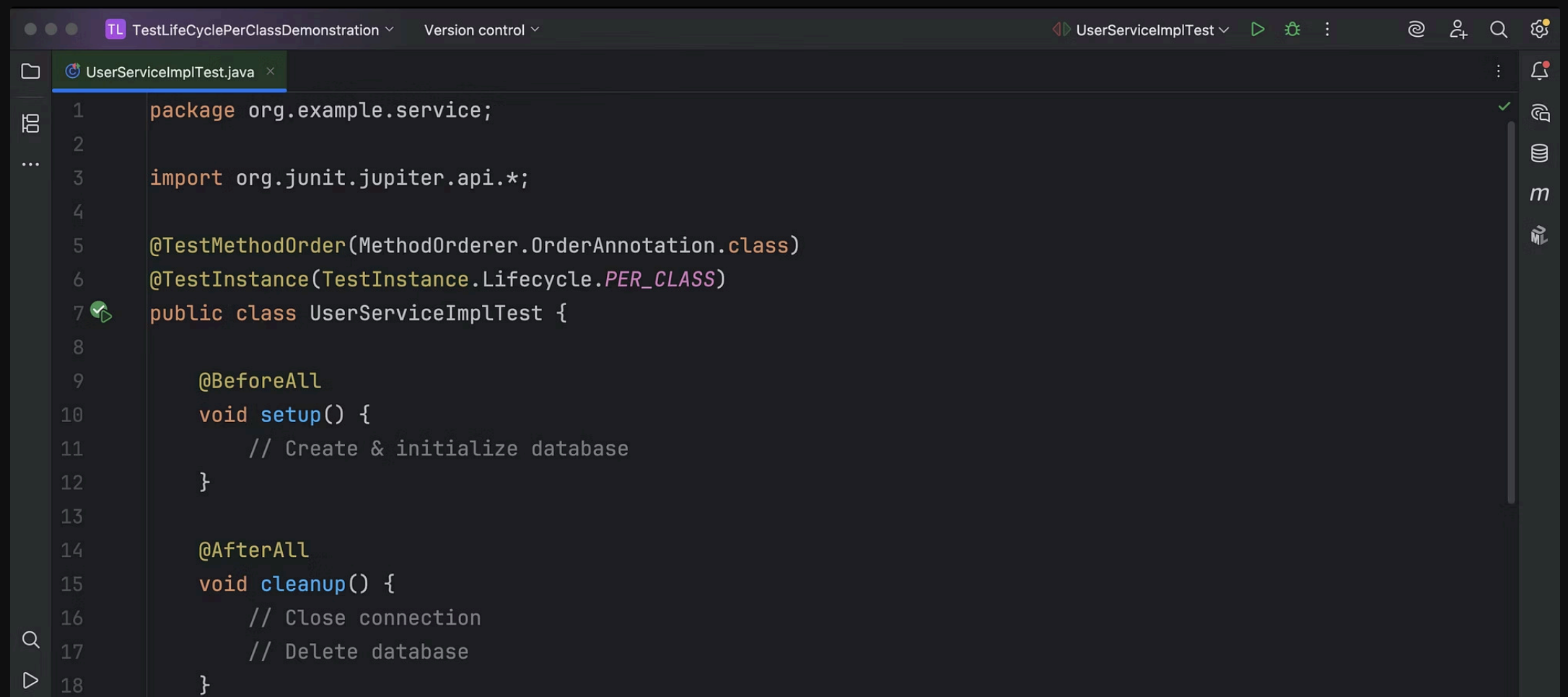
Because a **single instance** is used:

@BeforeAll (Lines 9-12)

@BeforeAll is not static

@AfterAll (Lines 14-17)

@AfterAll is not static



```
1 package org.example.service;
2
3 import org.junit.jupiter.api.*;
4
5 @TestMethodOrder(MethodOrderer.OrderAnnotation.class)
6 @TestInstance(TestInstance.Lifecycle.PER_CLASS)
7 public class UserServiceImplTest {
8
9     @BeforeAll
10     void setup() {
11         // Create & initialize database
12     }
13
14     @AfterAll
15     void cleanup() {
16         // Close connection
17         // Delete database
18     }
```

This differs from the default **PER_METHOD lifecycle**.

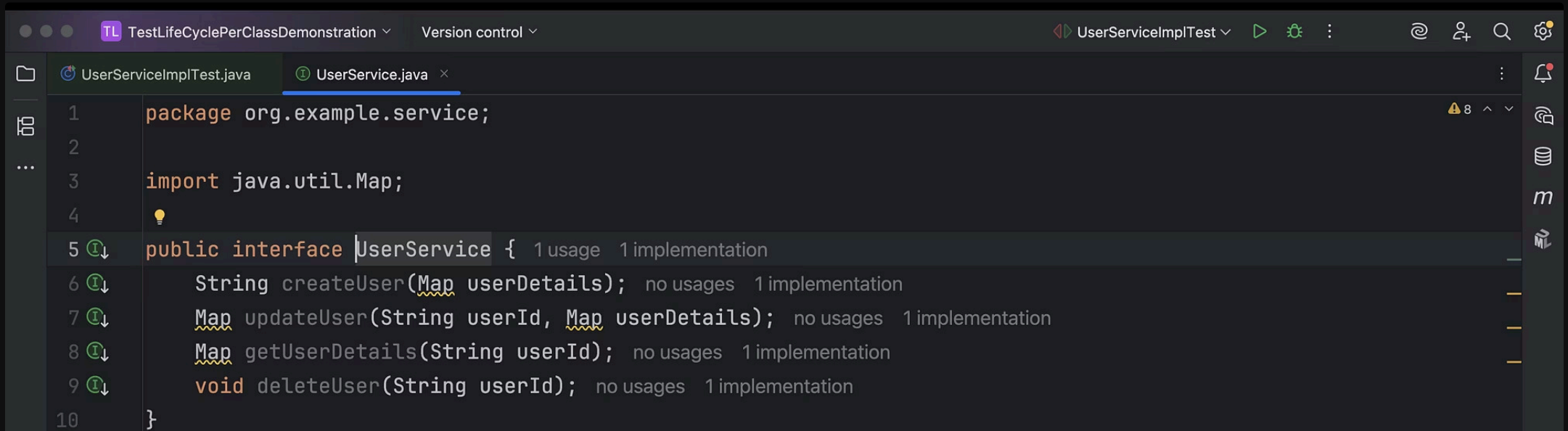
The **lifecycle** configuration **affects method signatures**.

Service Layer Under Test

Business logic is encapsulated in a **service class**.

A `UserService` **interface** defines **four methods**:

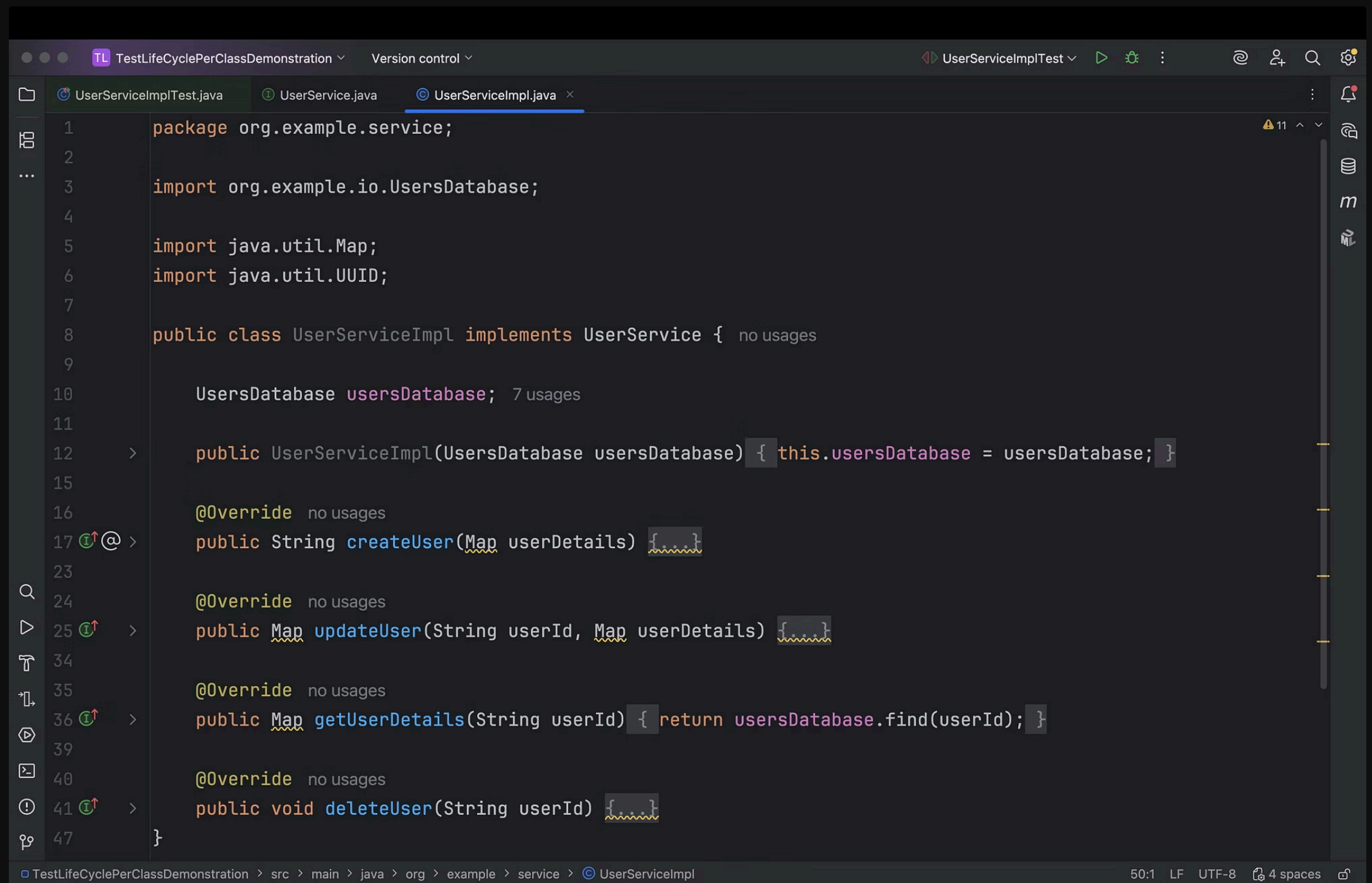
- Create, Update, Get, Delete



```
1 package org.example.service;
2
3 import java.util.Map;
4
5 public interface UserService { 1 usage 1 implementation
6     String createUser(Map userDetails); no usages 1 implementation
7     Map updateUser(String userId, Map userDetails); no usages 1 implementation
8     Map getUserDetails(String userId); no usages 1 implementation
9     void deleteUser(String userId); no usages 1 implementation
10 }
```

Service Layer Under Test (contd.)

The implementation of the **service layer** follows the **interface** (UserService) **contract**.



```
1 package org.example.service;
2
3 import org.example.io.UsersDatabase;
4
5 import java.util.Map;
6 import java.util.UUID;
7
8 public class UserServiceImpl implements UserService { no usages
9
10     UsersDatabase usersDatabase; 7 usages
11
12     > public UserServiceImpl(UsersDatabase usersDatabase) { this.usersDatabase = usersDatabase; }
13
14
15
16     @Override no usages
17     @ > public String createUser(Map userDetails) {
18
19
20
21
22
23
24     @Override no usages
25     @ > public Map updateUser(String userId, Map userDetails) {
26
27
28
29
30
31
32
33
34
35     @Override no usages
36     @ > public Map getUserDetails(String userId) { return usersDatabase.find(userId); }
37
38
39
40     @Override no usages
41     @ > public void deleteUser(String userId) {
42
43
44
45
46
47 }
```

The screenshot shows an IDE window with three tabs: 'TestLifeCyclePerClassDemonstration', 'UserService.java', and 'UserServiceImpl.java'. The 'UserServiceImpl.java' tab is active, displaying the implementation of the 'UserService' interface. The code includes package declarations, imports for 'UsersDatabase', 'Map', and 'UUID', and the implementation of methods 'createUser', 'updateUser', 'getUserDetails', and 'deleteUser'. The IDE interface includes a sidebar with icons for file explorer, search, and other tools, and a bottom status bar showing the file path and encoding details.

User Database Layer Abstraction

The **service depends** on a **user database interface**.

The database interface defines:



Initialize database



Close database



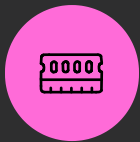
Save, update, find,
delete user

Dependency injection is used via constructor.

```
TestLifeCyclePerClassDemonstration Version control
UserServiceImplTest.java UserService.java UserServiceImpl.java UsersDatabase.java x
1 package org.example.io;
2
3 import java.util.Map;
4
5 public interface UsersDatabase { 4 usages 1 implementation
6     void init(); no usages 1 implementation
7     void close(); no usages 1 implementation
8     Map save(String userId, Map userDetails); 1 usage 1 implementation
9     Map update(String userId, Map user); 1 usage 1 implementation
10    Map find(String userId); 3 usages 1 implementation
11    void delete(String userId); 1 usage 1 implementation
12 }
```

User Database Layer Abstraction (contd.)

In-Memory Database Implementation



Custom Implementation

A custom in-memory implementation is provided.



Storage Mechanism

Users are **stored** in a **HashMap**.



No Real Database

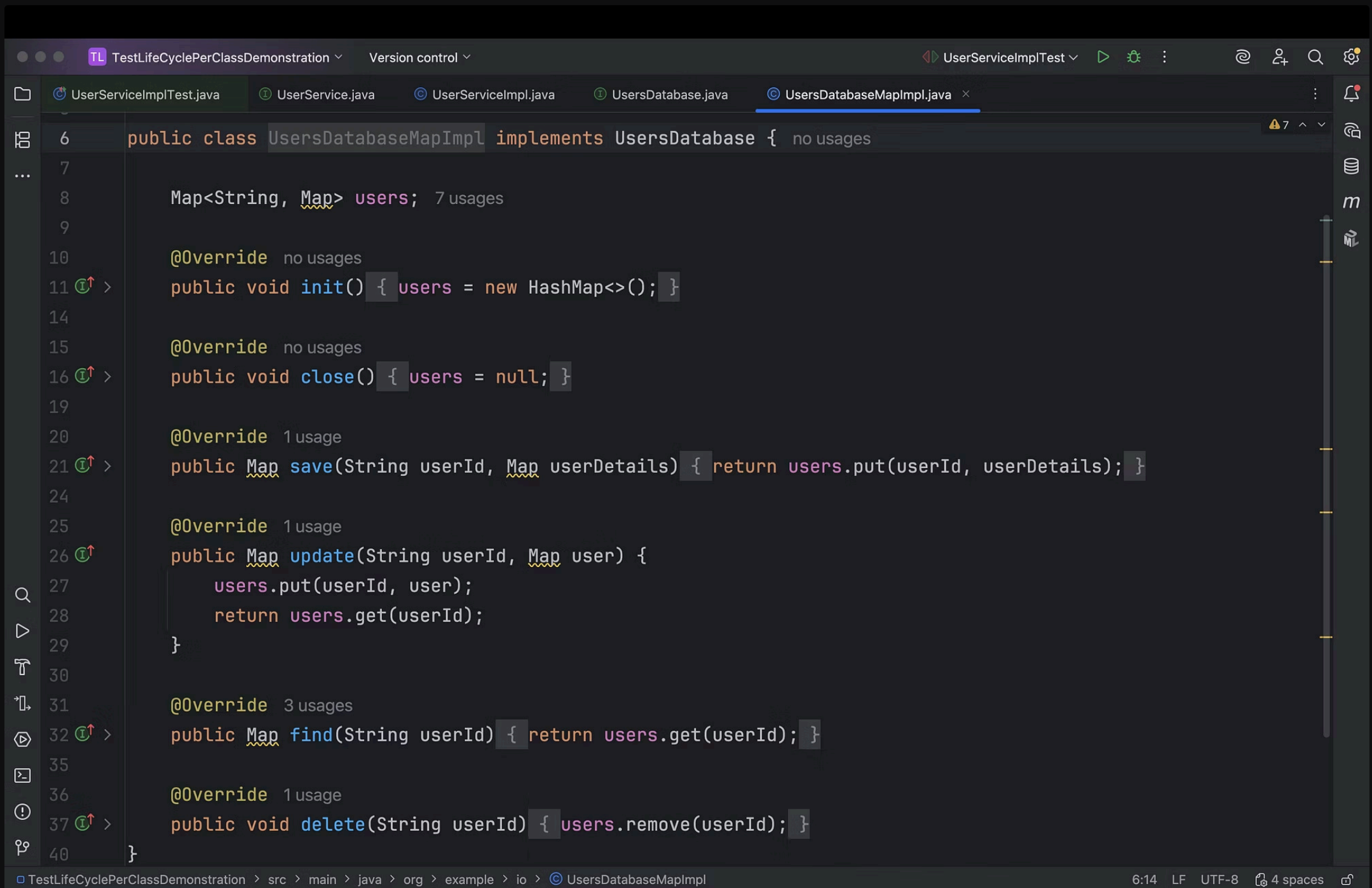
No real database is used in this demonstration.



Focus

Implementation details are **not** the **focus** here.

User Database Layer Abstraction (contd.)



The screenshot shows an IDE window with the file `UsersDatabaseMapImpl.java` open. The code implements the `UsersDatabase` interface. It includes a `Map` to store user details and several methods to manage this data. The IDE interface includes a top bar with project and file names, a left sidebar with navigation icons, and a bottom status bar showing the current file path and encoding.

```
6 public class UsersDatabaseMapImpl implements UsersDatabase { no usages
7
8     Map<String, Map> users; 7 usages
9
10    @Override no usages
11    public void init() { users = new HashMap<>(); }
14
15    @Override no usages
16    public void close() { users = null; }
19
20    @Override 1 usage
21    public Map save(String userId, Map userDetails) { return users.put(userId, userDetails); }
24
25    @Override 1 usage
26    public Map update(String userId, Map user) {
27        users.put(userId, user);
28        return users.get(userId);
29    }
30
31    @Override 3 usages
32    public Map find(String userId) { return users.get(userId); }
35
36    @Override 1 usage
37    public void delete(String userId) { users.remove(userId); }
40 }
```

TestLifeCyclePerClassDemonstration > src > main > java > org > example > io > UsersDatabaseMapImpl 6:14 LF UTF-8 4 spaces



Purpose of This Demonstration

1 Ordered Tests

Show how ordered tests work together.

2 State Sharing

Demonstrate state sharing with `PER_CLASS` lifecycle.

3 Realistic Testing

Illustrate realistic integration-style testing.

4 Preparation

Prepare for implementing the actual test methods.

Conclusion



Shared State

PER_CLASS enables shared state across test methods.



Ordered Execution

Ordered execution allows dependent tests to work correctly.



Integration Testing

This approach is useful for integration testing scenarios.



Configuration

Proper configuration is critical for predictable behavior.





Open Floor Discussion

Questions to Consider

- When should tests depend on each other?
- What risks come with shared test state?
- How can this approach affect test maintainability?